20 — Datenbanksysteme

Wozu Datenbanken? DBMS, Datenbankarten, Anomalien

HTW Berlin – Wirtschaftsingenieurwesen

WiSe 2026/27

Agenda

1. Wozu Datenbanken? – das Dateichaos
2. Warum nicht einfach Dateien? (Rückblick File I/O)
3. Datenbank $\neq$ DBMS – und das DBMS als Schicht
4. Die vier Datenbankarten
5. Excel stößt an Grenzen
6. Redundanz erzeugt Anomalien
7. **Klassiker: Anomalien-Audit** – live auditiert

Lernziel: Verstehen, was ein DBMS leistet, welche Datenbankarten es gibt – und **warum** eine chaotische Tabelle *mechanisch* zu Fehlern führt.

Heute: reine Theorie, kein Code – dafür nach jedem Konzept *gemeinsames Durchdenken* an der Folie und *Denkaufgaben* (Sie, Notebook 20).

Wozu Datenbanken? – das Dateichaos

Veggie Soles heute: ein Ordner voller Dateien

```
bestellung_2026-05-03_anna.txt
bestellung_2026-05-04_anna_v2_final.txt
kunden.xlsx   produkte.xlsx   lager_alt.xlsx
```

- *“Wie viel hat Anna Müller insgesamt bestellt?”* → alle Dateien einzeln öffnen
- Preis des Eco-Sneaker steht in `produkte.xlsx` **und** in jeder Bestellung → wer hat recht?
- zwei Mitarbeitende ändern `kunden.xlsx` gleichzeitig → eine Version verliert

Eine **Datenbank** speichert Daten *strukturiert an einem Ort* und beantwortet solche Fragen in Millisekunden – auch bei Millionen Einträgen.

Unter der Haube: zwei Programme, eine Datei

Der Lagerbestand als Textdatei (`bestand.txt` enthält `10`). Zwei Kassen verkaufen **gleichzeitig** je ein Paar:

Zeit	Kasse A	Kasse B	`bestand.txt`
t_1	liest `10`		`10`
t_2		liest `10`	`10`
t_3	schreibt `9`		`9`
t_4		schreibt `9`	`9` **statt 8!**

Lost Update: der Verkauf von Kasse A ist spurlos weg – kein Absturz, keine Fehlermeldung, nur stille Datenverfälschung. `open()` aus Foliensatz 12 weiß **nichts** von anderen Programmen.

Unter der Haube: Absturz mitten im Schreiben

```
with open("bestellungen.txt", "a") as f:
    f.write("101;Anna;Eco-Sneaker\n")  # 1) Puffer!
    f.write("101;89.95;2 Paar\n")      # 2) Crash?
```

- Rückblick Foliensatz 12: `write()` legt Daten erst in den **Puffer** – auf der Platte landen sie später
- Stromausfall/Absturz dazwischen → die Datei enthält eine **halbe Bestellung** (oder gar nichts)
- Hinterher **nicht erkennbar**: die Datei sieht völlig normal aus

Ein DBMS garantiert stattdessen **ganz oder gar nicht**: eine Änderung wird vollständig wirksam – oder verschwindet komplett (Fachwort: *Transaktion*, Details in NB 21).

Datenbank ≠ Datenbankmanagementsystem

	Was ist das?	Analogie Bibliothek
Datenbank	die gespeicherten Daten selbst	die Bücher im Regal
DBMS	die Software, die sie verwaltet	Katalog + Bibliothekar:in

Das **DBMS** kann, was die Daten allein nicht können:

- mehrere Nutzer:innen **gleichzeitig** bedienen
- Daten **konsistent** halten (Regeln erzwingen)
- **Zugriffsrechte** durchsetzen
- bei Absturz **nichts verlieren**

MySQL = das DBMS · “alle Bestellungen von Veggie Soles” = die Datenbank.

Unter der Haube: das DBMS als Schicht

```
Anwendung (Kasse, Web-Shop, Python-Skript)
    |   "neue Bestellung für Anna"
DBMS      prüft, koordiniert, sichert
    |   liest & schreibt geordnet
Dateien   auf der Festplatte
```

- **gleichzeitige Zugriffe**: koordiniert, sperrt gezielt statt ganze Datei
- **Konsistenz**: Prüfregeln erzwingen (Typen, eindeutige IDs)
- **Rechte**: wer darf was sehen und ändern
- **Ausfallsicherheit**: ganz-oder-gar-nicht statt halber Dateien

Wieder eine **Abstraktionsschicht** (vgl. E4: Betriebssystem über Hardware): die Anwendung redet nur mit dem DBMS – nie direkt mit Dateien.

Datenbanken stecken überall

System	Wofür	Beispiel
WWS – Warenwirtschaft	Lagerbestand in Echtzeit	Kasse scannt → Bestand sinkt
ERP – Enterprise Resource Planning	alle Abteilungen, eine Datenbasis	Buchhaltung, Personal, Produktion
CMS – Content Management	Webinhalte dynamisch	Wikipedia, Nachrichtenportale
CRM – Customer Relationship Mgmt	jeder Kundenkontakt nachvollziehbar	Mails, Anrufe, Bestellhistorie

✎ **Selbst durchdenken** – Notebook 20, Übung 1.1: drei Alltagssituationen finden, in denen Sie (unbemerkt) Datenbanken nutzen.

Die vier Datenbankarten – der Typ folgt dem Anwendungsfall

Die vier wichtigsten Datenbankarten — ein Typ pro Anwendungsfall

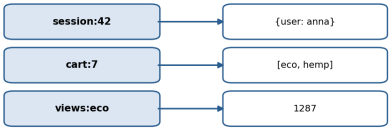
Relational · Tabellen

id	name	preis
1	Eco-Sneaker	89.95
2	Hemp-High	109.00
3	Bambus-Boot	135.50

feste Spalten, SQL, ACID

z. B. PostgreSQL, MySQL, SQLite

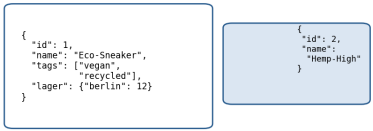
Key-Value · Schluessel → Wert



extrem schnell bei einfachen Lookups

z. B. Redis, DynamoDB

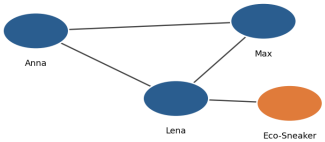
Dokument · flexible JSON-Dokumente



jedes Dokument darf andere Felder haben

z. B. MongoDB, CouchDB

Graph · Knoten & Kanten



effizient bei Beziehungsanalysen

z. B. Neo4j, Amazon Neptune

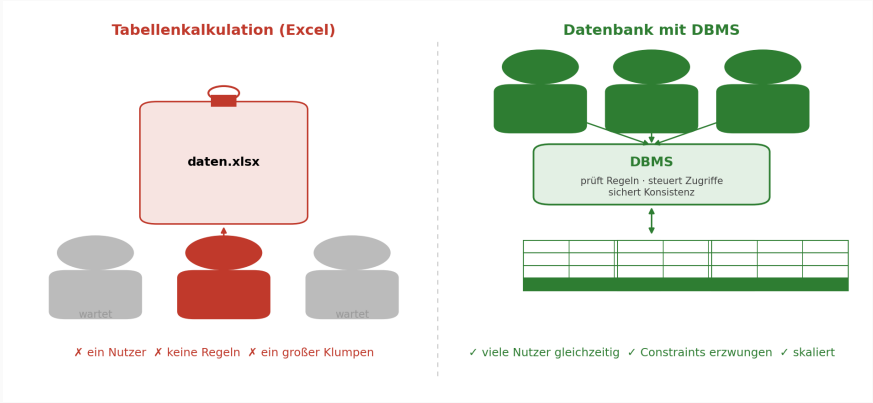
Welcher Typ passt zu welchem Problem?

Anforderung	passender Typ
feste Struktur, Verknüpfungen (Bank, Shop, ERP)	**relational** (Tabellen)
schnelle Lookups: Sessions, Caching	**Key-Value** (wie ein Dict)
flexible Datensätze (Produktkataloge)	**Dokument**
Beziehungen analysieren (soziale Netze)	**Graph**

- oft **Kombinationen**: relationale DB für Bestellungen *plus* Key-Value für Live-Tracking
- Veggie Soles → **relationale** Datenbank (klare Struktur) – unser Weg ab jetzt

✎ **Selbst durchdenken** – Notebook 20, Übung 2.1: vier Szenarien dem passenden Datenbanktyp zuordnen.

Excel stößt an Grenzen



✎ **Selbst durchdenken** – Notebook 20, Übung 3.1: vier Szenarien – Excel oder Datenbank? Entscheidung begründen.

Excel vs. Datenbank – konkret quantifiziert

Kriterium	Excel	Datenbank (DBMS)
Zeilen pro Tabelle	1.048.576 (fest, 2^{20})	praktisch unbegrenzt
gleichzeitig schreiben	**1** Person, Datei gesperrt	hunderte gleichzeitig
Prüfregeln	keine – jede Zelle nimmt alles	erzwungen (Typen, IDs)
Zugriffsrechte	ganze Datei oder nichts	pro Tabelle, pro Zeile
Absturzsicherheit	letzter Speicherstand	ganz-oder-gar-nicht

Für 50 Kontakte reicht Excel. Für ein Krankenhaus, einen Marktplatz – oder Veggie Soles im Wachstum: Datenbank.

Redundanz: eine Tabelle für alles

Bestellverwaltung “pragmatisch” – Kundendaten einfach in jede Zeile:

BestellNr	Kunde	E-Mail	Produkt	Preis
101	Anna Müller	anna@example.de	Eco-Sneaker	89.95
102	Max Schmidt	max@example.de	Hemp-High	109.00
103	Anna Müller	anna@example.de	Hemp-High	109.00
104	Anna Müller	anna@example.de	Eco-Sneaker	89.95

- Annas E-Mail steht **dreimal** da, der Eco-Sneaker-Preis **zweimal**
- **Redundanz** = dieselbe Information mehrfach gespeichert: kostet Speicher – und jede Kopie kann **auseinanderlaufen**

Unter der Haube: die Update-Anomalie, Schritt für Schritt

Anna bekommt eine neue E-Mail-Adresse. In der Tabelle von eben:

Schritt	Aktion	Zustand danach
1	Zeile 101 ändern	Zeilen 103, 104 noch alt
2	Zeile 103 ändern	Zeile 104 noch alt
3	Zeile 104 **vergessen**	zwei E-Mails für Anna
4	Abfrage *“Annas E-Mail?”*	**zwei Antworten** – welche stimmt?

Mechanik: eine Information mit n Kopien braucht n Änderungen. Jede vergessene Kopie erzeugt einen **Widerspruch** – und die Tabelle sieht weiterhin völlig normal aus.

Die vier Anomalie-Typen – Ursache und Wirkung

Anomalie	Auslöser (Ursache)	Wirkung
Redundanz	Kundendaten in jeder Bestellzeile	Speicher + Fehlerquelle
Update	Änderung an n Stellen nötig	vergessene Kopie $\rightarrow$ Widerspruch
Insert	neues Produkt ohne Bestellung	Zeile lässt sich nicht anlegen
Delete	letzte Bestellung storniert	Kundendaten verschwinden mit

▶ **Gemeinsam an der Folie** – wir spielen Insert und Delete an der Bestelltabelle durch: Was scheitert beim Eintragen? Was verschwindet beim Löschen?

Klassiker: Anomalien-Audit – die Aufgabe

Die komplette Bestellverwaltung von Veggie Soles – **eine** Tabelle:

BestellNr	Kunde	E-Mail	Produkt	Preis	Menge
101	Anna Müller	anna@example.de	Eco-Sneaker	89.95	1
102	Max Schmidt	max@example.de	Hemp-High	109.00	1
103	Anna Müller	anna@example.de	Hemp-High	109.00	2
104	Anna Müller	ana@example.de	Eco-Sneaker	89.95	1
105	Anna Müller	anna@example.de	Bambus-Boot	135.50	1
106	Anna Müller	anna@example.de	Eco-Sneaker	89.95	2

- Finden Sie je **ein Beispiel**: Redundanz · Update · Insert · Delete
- Ein **echter Widerspruch** steckt schon drin – wo?
- Transfer: welche *alltägliche Änderung* wird hier gefährlich?

Klassiker: Anomalien-Audit – Live-Audit

▶ **Live-Audit an der Folie/Tafel** – Referenz: `90_klassiker_loesung.md`
Teilziele:

1. Redundanz markieren: Was steht mehrfach da?
2. Den eingebauten Widerspruch aufspüren
3. Update-Anomalie durchspielen: Anna wechselt die E-Mail
4. Insert-Anomalie: neues Produkt ohne Bestellung eintragen?
5. Delete-Anomalie: Bestellung 102 stornieren – was verschwindet?

✎ **Zum Selberlösen** – Handout *“Klassiker 20: Anomalien-Audit”* (`Klassiker/20_Klassiker_Anomalien_Audit.pdf`). Vertiefung: Notebook 20, Abschlussaufgaben 3 und 4.

Cheat Card

Begriff	Kurz
Datenbank	strukturierte Sammlung von Daten
DBMS	Software-Schicht, die die Daten verwaltet
Datenbankarten	relational · Key-Value · Dokument · Graph
Redundanz	dieselbe Information mehrfach gespeichert
Anomalien	Update / Insert / Delete – Folgefehler der Redundanz
Klassiker heute	Anomalien-Audit: 4 Fehlertypen in einer Tabelle aufspüren

Faustregel: Daten mehrfach gespeichert oder mehrere Schreiber gleichzeitig $\longrightarrow$ Datei-Bastelei am Ende, DBMS nutzen.

Ausblick: Notebook 21 – Relationale Datenbanken

Unser Audit hat die Krankheit **diagnostiziert** – Notebook 21 liefert die Therapie:

- die Chaos-Tabelle in **mehrere verknüpfte Tabellen** zerlegen: jede Information genau **einmal**
- **Primärschlüssel** und **Fremdschlüssel**: Zeilen eindeutig machen und sauber verbinden
- Beziehungstypen 1:1, 1:n, n:m – danach (NB 21b) das **ER-Diagramm**: ein Datenmodell *zeichnen*, bevor man es baut
- ab Notebook 22 dann praktisch: die Abfragesprache **SQL**

Vorschau: Diese Begriffe sind Stoff der **nächsten** Vorlesungen – heute nur als Wegweiser gerahmt, noch nichts davon wird vorausgesetzt.

Heute (durch)gedacht

- ✓ Lost Update und halbe Dateien – warum Dateien allein nicht reichen
- ✓ Datenbank von DBMS unterschieden (Bücher vs. Bibliothekar:in)
- ✓ Das DBMS als Abstraktionsschicht über den Dateien
- ✓ Die vier Datenbankarten – Typ folgt Anwendungsfall
- ✓ Excel-Grenzen: Zeilenlimit, ein Schreiber, keine Prüfregeln
- ✓ Redundanz erzeugt Anomalien: Update / Insert / Delete
- ✓ **Klassiker Anomalien-Audit** live gesehen

✎ **Ihre Aufgaben bis zur nächsten Woche:**

- Klassiker **Anomalien-Audit** eigenständig lösen (Handout)
- Notebook 20: Übungen 1.1, 2.1, 3.1 und Abschlussaufgaben 1–4